

Tank Level Control with Real-Time Optimization: Problem Formulation and RL-RTO Architecture Development

November 22, 2025

Contents

1	Overview	2
2	Notation and Parameters	2
2.1	Time horizon and indices	2
2.2	Tank geometry and physical parameters	2
2.3	Model variables	3
3	Continuous-Time Model	3
3.1	Mass balance	3
3.2	Outflow relation	3
3.3	Complete dynamics	4
4	Operational Constraints (Continuous-time)	4
5	Backward-Euler Discretization and Discrete Model	4
5.1	Discrete variables (per time step k)	4
5.2	Discrete dynamics (Backward–Euler)	5
5.3	Discrete constraints	5
6	Two-Layer RTO-Control Architecture	5
6.1	Purpose and overview	5
6.2	RTO layer: steady-state optimization	6
6.3	Control layer: PI controller	6
6.4	Control layer: Model Predictive Control (MPC)	7
6.5	Optimal control with move suppression	7
6.6	Safety constraint enforcement	8
6.7	Execution loop	9
6.8	RL observations and reward	9
7	Notes on Dynamics	9
8	Summary	9

1 Overview

This document describes a simple tank level control problem in which a **Real-Time Optimization (RTO)** layer generates economically optimal setpoints for the tank level, and a **feedback controller** attempts to track those setpoints over discrete time steps.

The system consists of:

- A single **cylindrical tank** (3D geometry) with constant horizontal cross-sectional area A [m²] and varying liquid level $h(t)$ [m]. The liquid volume is $V(t) = A \cdot h(t)$ [m³].
- An input flow rate $F_{\text{in}}(t)$, which serves as the **manipulated variable** (MV).
- An output flow rate $F_{\text{out}}(t)$, which depends on the tank level $h(t)$ through a valve equation: $F_{\text{out}}(t) = k_v h(t)$.
- The controlled variable is the liquid level $h(t)$ in the tank.

Two-layer architecture:

- (1) **RTO layer:** Solves a simple linear economic optimization problem to determine the optimal steady-state level setpoint h_{sp} .
- (2) **Control layer:** A feedback controller (e.g., PI controller) manipulates $F_{\text{in}}(t)$ to drive $h(t)$ toward h_{sp} .

The dynamics are **linear** and discretized using the **backward Euler** method.

2 Notation and Parameters

2.1 Time horizon and indices

Continuous time horizon: $t \in [0, t_f]$.

Discrete grid (used in Section 5): $t_k = k \Delta t$, $k = 0, 1, \dots, N$ with $\Delta t = t_f/N$.

2.2 Tank geometry and physical parameters

Table 1: Tank geometry and physical parameters

Symbol	Description	Units / Value
A	Cross-sectional area of tank	m ²
k_v	Valve constant (outflow proportional to level)	m ² /s
h_{min}	Minimum allowable tank level (RTO)	m
h_{max}	Maximum allowable tank level	m
h_{safety}	Hard safety constraint on minimum level	m
$F_{\text{in,min}}$	Minimum input flow rate	m ³ /s
$F_{\text{in,max}}$	Maximum input flow rate	m ³ /s

2.3 Model variables

Table 2: Continuous-time model variables

Symbol	Description	Units
$h(t)$	Tank liquid level (controlled variable)	m
$F_{\text{in}}(t)$	Input flow rate (manipulated variable)	m ³ /s
$F_{\text{out}}(t)$	Output flow rate	m ³ /s

Table 3: Discrete-time model variables

Symbol	Description	Units
h_k	Tank liquid level at time step k	m
$F_{\text{in},k}$	Input flow rate at time step k	m ³ /s
$F_{\text{out},k}$	Output flow rate at time step k	m ³ /s

3 Continuous-Time Model

3.1 Mass balance

The tank is a 3D cylindrical vessel with:

- Cross-sectional area: A [m²] (constant)
- Liquid level: $h(t)$ [m] (time-varying)
- Liquid volume: $V(t) = A \cdot h(t)$ [m³]

The fundamental volumetric mass balance is:

$$\frac{dV}{dt} = F_{\text{in}}(t) - F_{\text{out}}(t) \quad (1)$$

Since $V(t) = A \cdot h(t)$ and A is constant:

$$A \frac{dh}{dt} = F_{\text{in}}(t) - F_{\text{out}}(t) \quad (2)$$

3.2 Outflow relation

Two outflow models are considered:

Linear outflow (simple valve). The outflow is proportional to the liquid level:

$$F_{\text{out}}(t) = k_v h(t) \quad (3)$$

where $k_v > 0$ is the valve constant [m²/s].

Nonlinear outflow (Torricelli's law). For gravity-driven flow at the tank bottom, the outflow is:

$$F_{\text{out}}(t) = k_v \sqrt{h(t)} \quad (4)$$

where $k_v > 0$ is a constant value. This introduces nonlinearity into the dynamics.

3.3 Complete dynamics

Linear model. Substituting the linear outflow relation into the mass balance yields:

$$\frac{dh}{dt} = \frac{1}{A} (F_{\text{in}}(t) - k_v h(t)) \quad (5)$$

This is a first-order linear system with $F_{\text{in}}(t)$ as input and $h(t)$ as output.

Nonlinear model. Substituting the nonlinear outflow relation Eq. (4) yields:

$$\frac{dh}{dt} = \frac{1}{A} (F_{\text{in}}(t) - k_v \sqrt{h(t)}) \quad (6)$$

This is a first-order **nonlinear** system.

4 Operational Constraints (Continuous-time)

Initial conditions. Can be set from measurements or steady-state values:

$$h(0) = h_0. \quad (7)$$

Level bounds.

$$h_{\min} \leq h(t) \leq h_{\max} \quad (8)$$

Input flow bounds.

$$F_{\text{in},\min} \leq F_{\text{in}}(t) \leq F_{\text{in},\max} \quad (9)$$

Output flow. The output flow is implicitly bounded by the level bounds:

$$F_{\text{out}}(t) = k_v h(t) \implies k_v h_{\min} \leq F_{\text{out}}(t) \leq k_v h_{\max} \quad (10)$$

5 Backward-Euler Discretization and Discrete Model

Let $k = 0, \dots, N$ with $t_k = k \Delta t$. Denote $h_k \approx h(t_k)$ (similarly for $F_{\text{in},k}$, $F_{\text{out},k}$). Backward-Euler for any state x reads

$$\left. \frac{dx}{dt} \right|_{t=t_k} \approx \frac{x_k - x_{k-1}}{\Delta t}. \quad (11)$$

5.1 Discrete variables (per time step k)

State: h_k ; Manipulated variable: $F_{\text{in},k}$; Output: $F_{\text{out},k}$.

5.2 Discrete dynamics (Backward–Euler)

Linear model. Applying backward Euler to the linear mass balance Eq. (5):

$$h_k = h_{k-1} + \Delta t \left[\frac{1}{A} (F_{\text{in},k} - k_v h_k) \right]. \quad (12)$$

Rearranging yields the explicit update form:

$$h_k = \frac{h_{k-1} + \frac{\Delta t}{A} F_{\text{in},k}}{1 + \frac{\Delta t k_v}{A}} = \alpha h_{k-1} + \beta F_{\text{in},k}, \quad (13)$$

where

$$\alpha = \frac{1}{1 + \frac{\Delta t k_v}{A}}, \quad \beta = \frac{\Delta t / A}{1 + \frac{\Delta t k_v}{A}}. \quad (14)$$

The discrete output flow is:

$$F_{\text{out},k} = k_v h_k. \quad (15)$$

Nonlinear model. Applying backward Euler to the nonlinear mass balance Eq. (6):

$$h_k = h_{k-1} + \frac{\Delta t}{A} (F_{\text{in},k} - k_v \sqrt{h_k}). \quad (16)$$

where:

$$F_{\text{out},k} = k_v \sqrt{h_k}. \quad (17)$$

For the nonlinear model we can use NLP solvers, such as IPOPT.

5.3 Discrete constraints

Bounds.

$$h_{\min} \leq h_k \leq h_{\max}, \quad (18)$$

$$F_{\text{in},\min} \leq F_{\text{in},k} \leq F_{\text{in},\max}. \quad (19)$$

Initial conditions.

$$h_0 = h_0^{\text{given}}. \quad (20)$$

6 Two-Layer RTO-Control Architecture

6.1 Purpose and overview

The system operates in a two-layer hierarchy:

- (1) **RTO layer (economic optimization):** Determines the economically optimal steady-state level setpoint h_{sp} by solving a simple linear optimization problem. The RTO layer assumes the system reaches steady state and minimizes an economic objective (e.g., cost of input flow).
- (2) **Control layer (dynamic regulation):** A feedback controller (e.g., PI controller) manipulates $F_{\text{in},k}$ to drive the actual level h_k toward the setpoint h_{sp} over discrete time steps.

6.2 RTO layer: steady-state optimization

Linear model. At steady state, $\frac{dh}{dt} = 0$, which implies:

$$F_{\text{in}}^{\text{ss}} = k_v h^{\text{ss}} \quad (21)$$

The RTO problem minimizes economic cost:

$$\min_{h^{\text{ss}}, F_{\text{in}}^{\text{ss}}} c F_{\text{in}}^{\text{ss}} \quad (22)$$

subject to:

$$F_{\text{in}}^{\text{ss}} - k_v h^{\text{ss}} = 0 \quad (\text{steady-state condition}) \quad (23)$$

$$h_{\text{min}} \leq h^{\text{ss}} \leq h_{\text{max}} \quad (24)$$

$$F_{\text{in},\text{min}} \leq F_{\text{in}}^{\text{ss}} \leq F_{\text{in},\text{max}} \quad (25)$$

where $c > 0$ is the cost coefficient for input flow.

Solution: Since the objective is to minimize $F_{\text{in}}^{\text{ss}} = k_v h^{\text{ss}}$, the optimal solution is:

$$h_{\text{sp}} = h^{\text{ss},*} = h_{\text{min}} \quad (26)$$

Nonlinear model. At steady state, $\frac{dh}{dt} = 0$, which implies:

$$F_{\text{in}}^{\text{ss}} = k_v \sqrt{h^{\text{ss}}} \quad (27)$$

The RTO problem is:

$$\min_{h^{\text{ss}}, F_{\text{in}}^{\text{ss}}} c F_{\text{in}}^{\text{ss}} \quad (28)$$

subject to:

$$F_{\text{in}}^{\text{ss}} - k_v \sqrt{h^{\text{ss}}} = 0 \quad (\text{steady-state condition}) \quad (29)$$

$$h_{\text{min}} \leq h^{\text{ss}} \leq h_{\text{max}} \quad (30)$$

$$F_{\text{in},\text{min}} \leq F_{\text{in}}^{\text{ss}} \leq F_{\text{in},\text{max}} \quad (31)$$

Solution: Since $F_{\text{in}}^{\text{ss}} = k_v \sqrt{h^{\text{ss}}}$ is monotonically increasing in h^{ss} , the optimal solution is:

$$h_{\text{sp}} = h^{\text{ss},*} = h_{\text{min}}, \quad F_{\text{in}}^{\text{ss}} = k_v \sqrt{h_{\text{min}}} \quad (32)$$

6.3 Control layer: PI controller

The controller tracks the setpoint h_{sp} by computing the manipulated variable $F_{\text{in},k}$ based on the error:

$$e_k = h_{\text{sp}} - h_k. \quad (33)$$

A **PI (Proportional-Integral)** controller is implemented in discrete incremental form:

$$\Delta u_k = K_c \left[e_k - e_{k-1} + \frac{\Delta t}{\tau_I} e_k \right], \quad (34)$$

$$F_{\text{in}}^k = F_{\text{in}}^{k-1} + \Delta u_k, \quad F_{\text{in},\text{min}} \leq F_{\text{in}}^k \leq F_{\text{in},\text{max}}, \quad (35)$$

where:

- K_c is the controller gain.
- τ_I is the integral time constant.
- $e_k = h_{\text{sp}} - h_k$ is the tracking error at time step k .
- Δu_k is the incremental change in the manipulated variable.

6.4 Control layer: Model Predictive Control (MPC)

As an alternative to PI control, a **Model Predictive Control (MPC)** strategy can be employed. MPC explicitly optimizes future control actions over a prediction horizon N by solving an online optimization problem at each time step.

MPC formulation. At time step k , given current state h_k and setpoint h_{sp} , solve:

$$\min_{F_{\text{in},k+1}, \dots, F_{\text{in},k+N}} \sum_{i=1}^N [Q(h_{k+i} - h_{\text{sp}})^2 + R(F_{\text{in},k+i} - F_{\text{in}}^{\text{ss}})^2] + Q_f(h_{k+N} - h_{\text{sp}})^2 \quad (36)$$

subject to:

$$h_{k+i+1} = \alpha h_{k+i} + \beta F_{\text{in},k+i+1}, \quad i = 0, \dots, N-1 \quad (37)$$

$$h_{k+i} \geq h_{\text{safety}}, \quad i = 1, \dots, N \quad (38)$$

$$F_{\text{in},\text{min}} \leq F_{\text{in},k+i} \leq F_{\text{in},\text{max}}, \quad i = 1, \dots, N \quad (39)$$

$$|F_{\text{in},k+i} - F_{\text{in},k+i-1}| \leq \Delta F_{\text{max}}, \quad i = 1, \dots, N \quad (40)$$

$$h_k = h_k^{\text{measured}} \quad (41)$$

where:

- N is the prediction horizon.
- $Q > 0$ is the state tracking weight (penalizes deviation from setpoint).
- $R \geq 0$ is the input weight (penalizes deviation from steady-state input and smooths control action).
- $Q_f \geq 0$ is the terminal state weight (enforces convergence at horizon end).
- $\Delta F_{\text{max}} > 0$ is the maximum input rate of change (limits actuator slew rate).
- $F_{\text{in}}^{\text{ss}} = k_v h_{\text{sp}}$ is the steady-state input corresponding to the setpoint.

Receding horizon implementation. Only the first control action $F_{\text{in},k+1}^*$ from the optimal solution is applied to the plant. At the next time step, the optimization is re-solved with updated measurements, implementing a **receding horizon** strategy.

6.5 Optimal control with move suppression

As an alternative to hard rate limits, **move suppression** can be included in the objective function as a soft penalty.

Formulation. Solve a single optimization problem over the entire horizon N (not receding):

$$\min_{F_{\text{in},1}, \dots, F_{\text{in},N}} \sum_{k=1}^N [c F_{\text{in},k} + Q(h_k - h_{\text{sp}})^2 + S(F_{\text{in},k} - F_{\text{in},k-1})^2] \quad (42)$$

subject to:

$$h_k = h_{k-1} + \frac{\Delta t}{A}(F_{\text{in},k} - k_v \sqrt{h_k}), \quad k = 1, \dots, N \quad (43)$$

$$F_{\text{out},k} = k_v \sqrt{h_k}, \quad k = 1, \dots, N \quad (44)$$

$$h_k \geq h_{\text{safety}}, \quad k = 1, \dots, N \quad (45)$$

$$F_{\text{in},\min} \leq F_{\text{in},k} \leq F_{\text{in},\max}, \quad k = 1, \dots, N \quad (46)$$

$$|F_{\text{in},k} - F_{\text{in},k-1}| \leq \Delta F_{\max}, \quad k = 1, \dots, N \quad (47)$$

$$h_0 = h_0^{\text{given}}, \quad F_{\text{in},0} = k_v \sqrt{h_0} \quad (48)$$

where:

- $c > 0$ is the economic cost of flow.
- $Q > 0$ is the tracking weight (penalizes deviation from RTO setpoint).
- $S \geq 0$ is the move suppression weight (penalizes input rate of change).
- h_{sp} is computed by the RTO layer as $h_{\text{sp}} = h_{\min}$.

Key differences from receding horizon MPC.

- **Single optimization:** Solves for the entire trajectory at once, not re-optimized at each step.
- **Nonlinear dynamics:** Uses the nonlinear outflow model Eq. (44).
- **Move suppression in objective:** Rate of change is penalized softly via S term, in addition to hard rate limits Eq. (47).
- **Economic cost:** Includes flow cost $c F_{\text{in},k}$ directly in the objective.

6.6 Safety constraint enforcement

To ensure that the tank level never drops below a critical safety threshold h_{safety} during transient operation, we enforce a **hard state constraint**:

$$h_k \geq h_{\text{safety}} \quad \forall k \quad (49)$$

This constraint must be satisfied at all time steps, even during aggressive setpoint tracking or disturbances. To enforce this, we compute the **minimum required input flow** to prevent constraint violation in the next time step.

From the discrete dynamics Eq. (13):

$$h_{k+1} = \alpha h_k + \beta F_{\text{in},k+1} \quad (50)$$

Requiring $h_{k+1} \geq h_{\text{safety}}$ yields:

$$\alpha h_k + \beta F_{\text{in},k+1} \geq h_{\text{safety}} \quad (51)$$

$$F_{\text{in},k+1} \geq \frac{h_{\text{safety}} - \alpha h_k}{\beta} \quad (52)$$

Define the **safety-constrained minimum input flow**:

$$F_{\text{in},k+1}^{\text{safe}} = \frac{h_{\text{safety}} - \alpha h_k}{\beta} \quad (53)$$

The control action from the PI controller is then saturated to satisfy both input bounds and the safety constraint:

$$F_{\text{in}}^k = \left(\max \left(F_{\text{in}}^k, F_{\text{in},\min}, F_{\text{in}}^{\text{safe},k} \right), \min \left(F_{\text{in}}^k, F_{\text{in},\max} \right) \right) \quad (54)$$

6.7 Execution loop

- (1) RTO computes h_{sp} every Δt_{RTO} .
- (2) Controller (PI or MPC) adjusts F_{in} to track h_{sp} :
 - PI: Uses feedback control law Eqs. (34) and (35).
 - MPC: Solves optimization problem Eq. (36)–Eq. (41) and applies first control action.
- (3) Plant (tank) evolves under the discrete dynamics Eq. (13).
- (4) Measurements of h_k feed back to the control layer.

6.8 RL observations and reward

For future RL-based RTO replacement, the observation and reward structure can be defined as:

$$s_k = [h_k, e_k, F_{\text{in},k}], \quad (55)$$

$$r_k = -c F_{\text{in},k} - \lambda_1 (h_k - h_{\text{sp}})^2 - \lambda_2 \max(0, h_{\text{safety}} - h_k), \quad (56)$$

where λ_1 penalizes tracking error and λ_2 penalizes safety violations.

7 Notes on Dynamics

All physical plant dynamics reside in the lower (environment) layer. The upper RTO layer contains a steady-state or simplified predictive model used for optimization. An RL policy can replace repeated online optimization by directly mapping observed states to safe setpoints, learning from interaction with the true plant dynamics.

8 Summary

This document presents a tank level control problem with:

- **Two dynamics models:**
 - Linear: $\frac{dh}{dt} = \frac{1}{A}(F_{\text{in}} - k_v h)$
 - Nonlinear: $\frac{dh}{dt} = \frac{1}{A}(F_{\text{in}} - k_v \sqrt{h})$ (Torricelli’s law)
- Backward–Euler discretization (explicit for linear, implicit for nonlinear).
- Two-layer architecture: RTO layer generates economic setpoints, control layer tracks them.

- Simple economic objective: minimize input flow cost (solution: $h_{\text{sp}} = h_{\text{min}}$).
- **Hard safety constraint:** $h_k \geq h_{\text{safety}}$ enforced at all times.
- **Three control strategies:**
 - **PI control:** Computationally efficient feedback control for linear model.
 - **Receding horizon MPC:** Optimization-based predictive control with explicit tracking penalty and hard rate limits (linear model).
 - **Optimal control with move suppression:** Single-shot optimization over entire horizon with nonlinear dynamics, combining economic cost, tracking penalty, and move suppression in objective (Pyomo + IPOPT).
- Structured for future RL-based RTO replacement.

The implementation demonstrates reactive feedback control (PI), receding horizon predictive control (MPC), with nonlinear dynamics, providing a toy example for comparison with Reinforcement Learning (RL) approaches.